

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 1
по дисциплине «Анализ алгоритмов»

Тема Вычисление процессорного времени реализации различных алгоритмов сортировки.

Студент Одинцов Е.В.

Группа ИУ7-63БВ

Преподаватели Волкова Л.Л.

Москва, 2025

Содержание

ВВЕДЕНИЕ **4**

1 Аналитическая часть **5**

2 Технологическая часть **6**

3 Исследовательская часть **11**

3.1 Сравнение алгоритмов по времени работы их реализаций 12

ЗАКЛЮЧЕНИЕ **15**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **16**

ВВЕДЕНИЕ

В данном отчете представлены результаты исследования эффективности трех алгоритмов сортировки: сортировки пузырьком (Bubble Sort), сортировки вставками (Insertion Sort) и сортировки Шелла (Shell Sort). Цель исследования — сравнить время выполнения реализаций этих алгоритмов на массивах различного размера и при различных начальных условиях (случайный порядок элементов, отсортированный в порядке возрастания, отсортированный в порядке убывания).

1 Аналитическая часть

Алгоритмы сортировки являются одной из базовых тем в компьютерных науках. Эффективность алгоритма сортировки может значительно варьироваться в зависимости от его реализации и начальных данных. В данном исследовании рассматриваются три алгоритма.

— **Сортировка пузырьком** — алгоритм, который многократно проходит по массиву, сравнивая соседние элементы и меняя их местами, если они находятся в неправильном порядке.

— **Сортировка вставками** — алгоритм, который строит отсортированную последовательность, по одному элементу за раз, вставляя каждый новый элемент в правильное положение.

— **Сортировка Шелла** — улучшенный вариант сортировки вставками, который сначала сортирует элементы, находящиеся далеко друг от друга, и постепенно уменьшает расстояние между сравниваемыми элементами.

2 Технологическая часть

Для проведения исследования была разработана программа на языке C++, которая измеряет время выполнения каждого алгоритма на массивах различного размера. Программа генерирует три типа массивов:

— Случайный массив.

— Массив, отсортированный в порядке возрастания (лучший случай для некоторых алгоритмов).

— Массив, отсортированный в порядке убывания (худший случай для некоторых алгоритмов).

Для каждого массива измеряется время выполнения каждого алгоритма сортировки, и результаты записываются в файл CSV.

Программа использует высокоточный таймер Windows для измерения времени выполнения, используются методы QueryPerformanceCounter [6]. Каждый алгоритм запускается пять раз для каждого массива, и вычисляется среднее время выполнения. Результаты сохраняются в файл `sorting_times.csv`, который затем может быть использован для анализа.

В листинге 2.1 представлена реализация алгоритмов.

Листинг 2.1 — Код программы

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <windows.h>
#include <fstream>

LARGE_INTEGER getPerformanceCounter() {
    LARGE_INTEGER count;
    QueryPerformanceCounter(&count);
    return count;
}

double getTimeDiff(LARGE_INTEGER start, LARGE_INTEGER end,
   LARGE_INTEGER frequency) {
    return static_cast<double>(end.QuadPart - start.QuadPart) /
       frequency.QuadPart;
}
```

```
void BubbleSort(std::vector<int> &array) {
    for (size_t i = 0; i < array.size() - 1; ++i) {
        for (size_t j = 0; j < array.size() - 1 - i; ++j) {
            if (array[j] > array[j + 1]) {
                std::swap(array[j], array[j + 1]);
            }
        }
    }
}

void InsertionSort(std::vector<int> &array) {
    for (size_t step = 1; step < array.size(); ++step) {
        int elem = array[step];
        int j = step - 1;
        while (j >= 0 && array[j] > elem) {
            array[j + 1] = array[j];
            j--;
        }
        array[j + 1] = elem;
    }
}

void ShellSort(std::vector<int> &array) {
    int N = array.size();
    for (int gap = N / 2; gap > 0; gap /= 2) {
        for (int i = gap; i < N; i++) {
            int temp = array[i];
            int j;
            for (j = i; j >= gap && array[j - gap] > temp; j -=
               gap) {
                array[j] = array[j - gap];
            }
            array[j] = temp;
        }
    }
}

std::vector<int> generateRandomArray(int size) {
    std::vector<int> array(size);
```

```
    for (int &num: array) {
        num = rand() % 100000; // Генерим от 0 до 999999
    }
    return array;
}

std::vector<int> ASC(const std::vector<int> &array) {
    std::vector<int> sorted = array;
    std::sort(sorted.begin(), sorted.end());
    return sorted;
}

std::vector<int> DESC(const std::vector<int> &array) {
    std::vector<int> sorted = array;
    std::sort(sorted.begin(), sorted.end(), std::greater<>());
    return sorted;
}

void printArray(const std::vector<int> &array) {
    for (int i = 0; i < array.size(); ++i) {
        std::cout << array[i] << " ";
    }
    std::cout << std::endl;
}

double measureSortingTime(void (*sortFunction)(std::vector<int>
   &), std::vector<int> array) {
    LARGE_INTEGER frequency;
    QueryPerformanceFrequency(&frequency);

    double totalTime = 0;
    int runs = 5;
    for (int i = 0; i < runs; ++i) {
        std::vector<int> temp = array;
        //printArray(temp);
        LARGE_INTEGER start = getPerformanceCounter();
        sortFunction(temp);
        LARGE_INTEGER end = getPerformanceCounter();
        totalTime += getTimeDiff(start, end, frequency);
    }
```

```
    return totalTime / runs;
}

int main() {
    srand(static_cast<unsigned int>(time(0)));

    std::vector<int> sizes = {100, 150, 200, 250, 300};
    std::ofstream file("sorting_times.csv");
    file << "Size,Case,BubbleSort,InsertionSort,ShellSort\n";

    for (int size: sizes) {
        std::vector<int> randArray = generateRandomArray(size);
        std::vector<int> bestArray = ASC(randArray);
        std::vector<int> worstArray = DESC(randArray);

        double timeBubbleRand = measureSortingTime(BubbleSort,
           randArray);
        double timeInsertionRand = measureSortingTime(
           InsertionSort, randArray);
        double timeShellRand = measureSortingTime(ShellSort,
           randArray);

        double timeBubbleBest = measureSortingTime(BubbleSort,
           bestArray);
        double timeInsertionBest = measureSortingTime(
           InsertionSort, bestArray);
        double timeShellBest = measureSortingTime(ShellSort,
           bestArray);

        double timeBubbleWorst = measureSortingTime(BubbleSort,
           worstArray);
        double timeInsertionWorst = measureSortingTime(
           InsertionSort, worstArray);
        double timeShellWorst = measureSortingTime(ShellSort,
           worstArray);

        file << size << ",Random," << timeBubbleRand << "," <<
           timeInsertionRand << "," << timeShellRand << "\n";
        file << size << ",Best," << timeBubbleBest << "," <<
           timeInsertionBest << "," << timeShellBest << "\n";
```

```
        file << size << ",Worst," << timeBubbleWorst << "," <<
           timeInsertionWorst << "," << timeShellWorst << "\n";

        std::cout << "Size " << size << " done.\n";
    }

    file.close();
    std::cout << "Data saved to sorting_times.csv\n";

    return 0;
}
```

3 Исследовательская часть

Результаты исследования представлены в таблице 3.1. Время выполнения каждого алгоритма измерялось в секундах. Данные показывают, что алгоритм сортировки Шелла демонстрирует наилучшую производительность во всех случаях, за ним следуют сортировка вставками и сортировка пузырьком. Используются следующие обозначения характера заполнения массивов:

Best — в случае, если заданные массивы отсортированы по возрастанию;

Random — в случае, если элементы в заданных массивах расположены в случайном порядке;

Worst — в случае, если заданные массивы отсортированы по убыванию.

Таблица 3.1 — Время выполнения алгоритмов сортировки (в секундах)

Размер массива	**Тип массива**	**Bubble Sort**	**Insertion Sort**	**Shell Sort**
100	Best	0.0000141	0.0000004	0.0000015
	Random	0.0000341	0.0000073	0.0000052
	Worst	0.0000409	0.0000137	0.0000028
150	Best	0.0000442	0.0000009	0.0000044
	Random	0.0001363	0.0000233	0.0000108
	Worst	0.0000941	0.0000295	0.0000049
200	Best	0.0000525	0.0000007	0.0000034
	Random	0.0001808	0.0000437	0.0000140
	Worst	0.0001717	0.0000523	0.0000063
250	Best	0.0000907	0.0000009	0.0000044
	Random	0.0002662	0.0000700	0.0000219
	Worst	0.0002700	0.0000777	0.0000084
300	Best	0.0001048	0.0000010	0.0000052
	Random	0.0002670	0.0000591	0.0000216
	Worst	0.0003341	0.0001034	0.0000127

Анализ результатов

— **Сортировка пузырьком (Bubble Sort)**: Этот алгоритм показал наихудшие результаты во всех случаях. Время выполнения увеличивается с ростом размера массива, особенно заметно в худшем случае (отсортированный в порядке убывания массив). Например, для массива размером 300 элементов в худшем случае время выполнения составило 0.0003341 секунды.

— **Сортировка вставками (Insertion Sort)**: Этот алгоритм оказался быстрее сортировки пузырьком, особенно в лучшем случае (отсортированный массив), где время выполнения составило всего 1 мкс для массива размером 300 элементов. Однако в худшем случае его производительность также снижается.

— **Сортировка Шелла (Shell Sort)**: Этот алгоритм показал наилучшие результаты во всех случаях. Даже для массива размером 300 элементов в худшем случае время выполнения составило всего 13 мкс, что значительно быстрее, чем для реализаций двух других алгоритмов.

Выводы

— Сортировка Шелла является наиболее эффективной из трех рассмотренных алгоритмов, особенно на больших массивах.

— Сортировка вставками показывает хорошие результаты в лучшем случае, но её производительность значительно снижается в худшем случае.

— Сортировка пузырьком является наименее эффективной и не рекомендуется для использования на больших массивах.

3.1 Сравнение алгоритмов по времени работы их реализаций

На рисунках 3.1–3.3 представлены результаты замеров процессорного времени, потраченного на работу реализаций вышеуказанных алгоритмов.

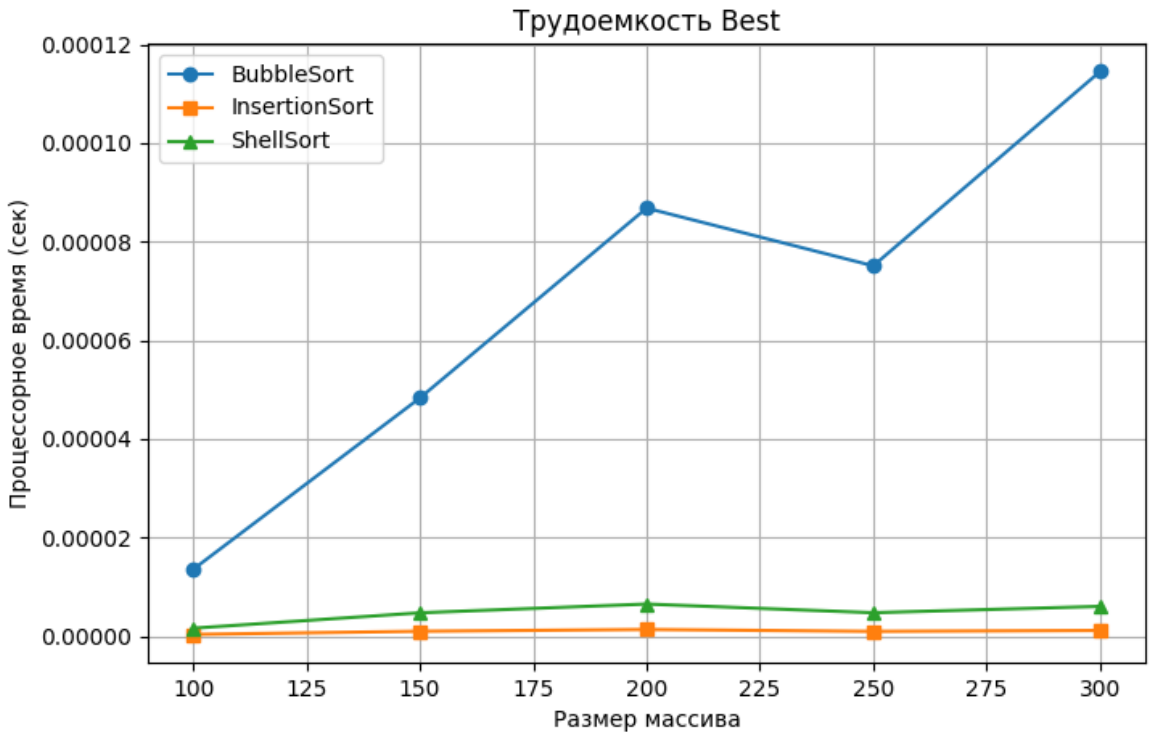


Рисунок 3.1 — Лучший случай

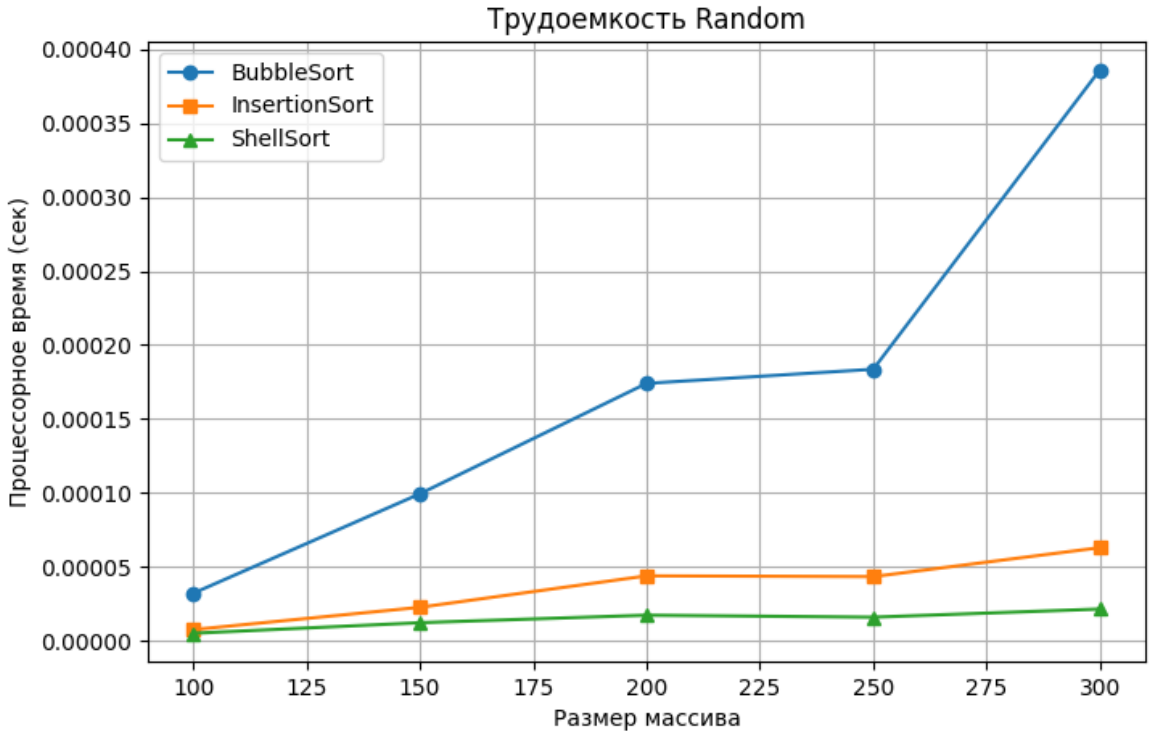


Рисунок 3.2 — Произвольный случай

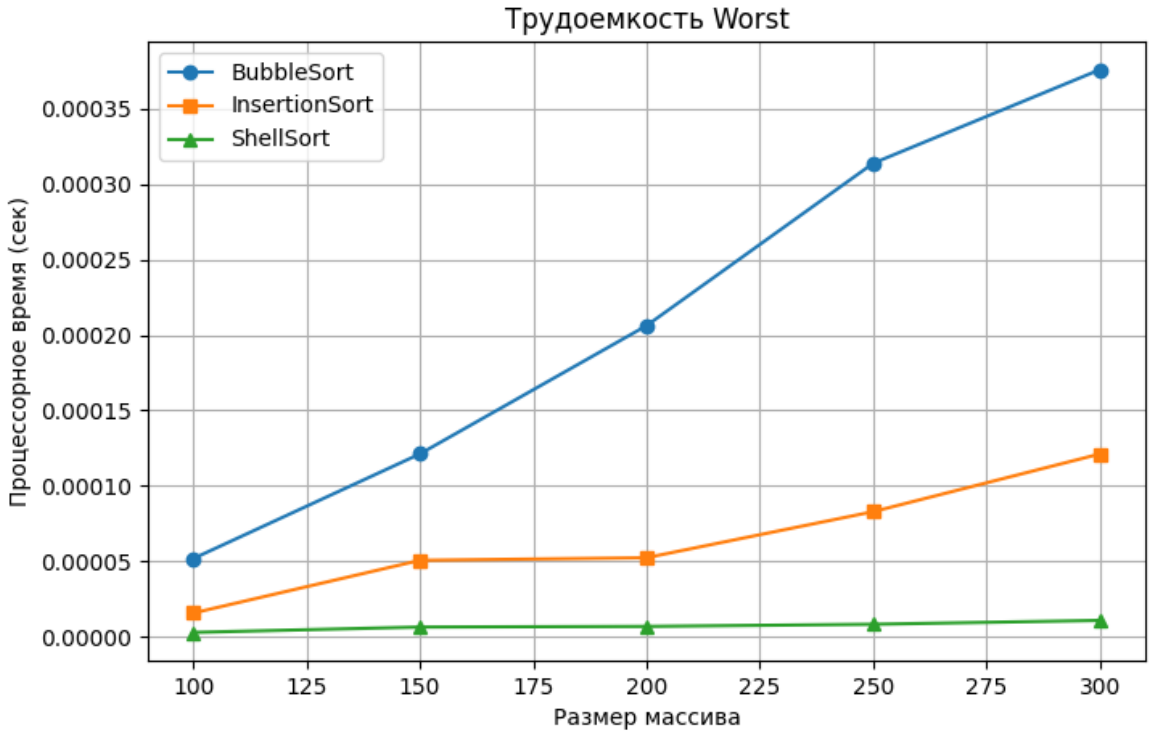


Рисунок 3.3 — Худший случай

ЗАКЛЮЧЕНИЕ

Исследование показало, что сортировка Шелла является наиболее эффективной из трех рассмотренных алгоритмов, особенно на больших массивах. Сортировка пузырьком и сортировка вставками показывают схожую производительность, но сортировка вставками немного быстрее в большинстве случаев. В худшем случае (массив, отсортированный в порядке убывания) все алгоритмы показывают наихудшие результаты, но сортировка Шелла остается наиболее эффективной.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ульянов М.В. Ресурсно-эффективные компьютерные алгоритмы. — М.: ФИЗМАТЛИТ, 2008. — 304 с.

2. Джеффри Макконнел. Анализ алгоритмов. — М.: Техносфера, 2023. — 416 с.

3. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. — М.: Вильямс, 2015. — 1328 с.

4. Седжвик Р. Алгоритмы на C++. — М.: Вильямс, 2019. — 1056 с.

5. Кнут Д.Э. Искусство программирования. Том 3. Сортировка и поиск. — М.: Диалектика-Вильямс, 2019. — 832 с.

6. Microsoft. QueryPerformanceCounter function. Доступно: `https://learn.microsoft.com/ru-ru/windows/win32/api/profileapi/nf-profileapi-queryperformancecounter`. [Дата обращения: 22.03.2025].